

Practical 1 - Introduction to MATLAB and floating-point arithmetic

Xiru Zhang

Year 2023/2024

Contents

1	Exercise 1	2
1.1	What should the program display	2
1.2	Program display	2
1.3	Explain the graph	2
2	Exercise 2	2
2.1	Calculate the integral in double precision	2
2.2	Enclosure of function $x^n e^{-x}$ and integral I_n and comparison	3
2.3	Deduce of I_n	4
2.4	Calculate I_n	4
2.5	Comparison of results	4
3	Exercise 3	5
3.1	Efficiency of Matlab and BLAS	5
3.2	Matrices multiplication	6
4	Exercise 4	7
4.1	LUP decomposition with partial pivoting strategy	7
4.2	Test of the implementation	8
4.3	Comparison with command lu	9

1 Exercise 1

1.1 What should the program display

The program takes the square root of the number 4, 52 times in succession, and then squares the resulting number 52 times. Theoretically, these two sets of operations should return the original value, which is 4.

1.2 Program display

Due to precision limitations and rounding errors in computer floating-point arithmetic, the final result is not 4 but approximately 2.718281808182473. This value is close to the mathematical constant e .

1.3 Explain the graph

The graph displays the impact of floating-point arithmetic on computational accuracy. The red dashed line represents the ideal outcome, where the output matches the input, thus $y = x$. The black dots, however, deviate from this line, showing the errors that arise from performing the same operation multiple times due to the limitations of floating-point representation in computers. The three separate lines of black dots indicate varying degrees of error at different scales.

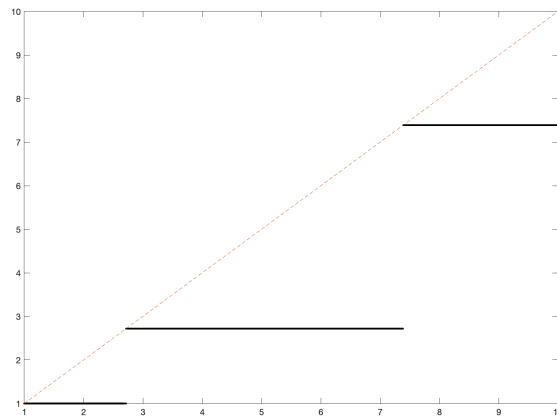


Figure 1: Higham function output (black dots) versus $y=x$ (dashed line), showing accuracy

2 Exercise 2

2.1 Calculate the integral in double precision

Listing 1: calculate_integral function

```
function I_n = calculate_integral(n)
n = double(n);
```

```

e_neg1 = double(exp(-1));
if n == 0
    I_n = 1 - e_neg1;
else
    I_prev = calculate_integral(n - 1);
    I_n = -e_neg1 + double(n) * I_prev;
end
end

```

2.2 Enclosure of function $x^n e^{-x}$ and integral I_n and comparison

The function $x^n e^{-x}$ on the interval $[0, 1]$ has a lower bound of 0 (since both x^n and e^{-x} are non-negative for x in $[0, 1]$) and an upper bound of 1 for x^n (since $x \leq 1$ on $[0, 1]$, thus $x^n \leq 1$ for non-negative n) and 1 for e^{-x} (since $e^{-x} \leq 1$ for $x \geq 0$). Therefore, we can estimate that:

$$0 \leq x^n e^{-x} \leq 1$$

for x in $[0, 1]$.

Based on this, the integral I_n , which is the area under the curve of $x^n e^{-x}$ from 0 to 1, also has an upper bound. Since the maximum value of $x^n e^{-x}$ on $[0, 1]$ is 1, the integral I_n over this interval must be less than or equal to the integral of 1 over the same interval, which is simply the length of the interval:

$$0 \leq I_n = \int_0^1 x^n e^{-x} dx \leq \int_0^1 1 dx = 1$$

So, for any n , the value of I_n is between 0 and 1. The numerical results obtained:

- $I_0 = 0.6321$
- $I_1 = 0.2642$
- $I_2 = 0.1606$
- $I_3 = 0.1139$
- $I_4 = 0.0878$
- $I_5 = 0.0713$

fit within these bounds, as expected. As n increases, I_n decreases within this range, which aligns with the behavior of the function $x^n e^{-x}$ where the exponential decay has a stronger effect as n gets larger.

2.3 Deduce of I_n

Given: $I_n = -e^{-1} + nI_{n-1}$ We need to express I_n in terms of I_{n+1} . To do this, we can apply the recurrence relation to I_{n+1} :

$$I_{n+1} = -e^{-1} + (n+1)I_n$$

Now, solve this equation for I_n :

$$I_n = \frac{I_{n+1} + e^{-1}}{n+1}$$

This is the recurrence relation expressing I_n in terms of I_{n+1} .

2.4 Calculate I_n

Listing 2: calculate_ I_n function

```
function In = calculate_In(n)
    m = 50; % Given m value
    In_plus_m = 12; % Given initial value for In+m
    e_inv = exp(-1); % Calculate e^(-1) only once
                     % for efficiency

    % Compute In by iteratively applying the
    % recurrence relation backward
    for k = (n+m):-1:(n+1)
        In = (In_plus_m + e_inv) / k;
        In_plus_m = In; % Update In_plus_m for the
                        % next iteration
    end
end
```

2.5 Comparison of results

	$n = 5$	$n = 10$
$m = 10$	0.07130217920875342	0.03646133464197853
$m = 20$	0.07130217810980316	0.03646133462410727
$m = 50$	0.07130217810980316	0.03646133462410727
$m = 100$	0.07130217810980316	0.03646133462410727
Exact value	0.071302178109803159860	0.036461334624107272383

Table 1: Approximate values of I_n for different m and comparison with exact values

We can evaluate the accuracy and stability of the recursive method by observing the differences between the computed results and the exact values. Generally, as m increases, which means using larger backward steps in the recursion, you would expect the approximate values of I_n to get closer to the exact values. This is because the recursive relation provides a way to gradually and accurately compute I_n from the known value of I_{n+m} .

3 Exercise 3

3.1 Efficiency of Matlab and BLAS

Algorithm 1 Column-Oriented Sum of Absolute Values

```

1:  $m \leftarrow$  number of rows of  $A$ 
2:  $n \leftarrow$  number of columns of  $A$ 
3:  $s \leftarrow$  vector of zeros of length  $m$ 
4: for  $i \leftarrow 1$  to  $m$  do
5:    $sum \leftarrow 0$ 
6:   for  $j \leftarrow 1$  to  $n$  do
7:      $sum \leftarrow sum + |A[i, j]|$ 
8:   end for
9:    $s[i] \leftarrow sum$ 
10: end for

```

Algorithm 2 BLAS-Based Sum of Absolute Values

```

1:  $m \leftarrow$  number of rows of  $A$ 
2:  $s \leftarrow$  vector of zeros of length  $m$ 
3: for  $i \leftarrow 1$  to  $m$  do
4:    $s[i] \leftarrow \text{norm}(A[i, :], 1)$ 
5: end for

```

The average run time for both algorithms increases with the size of the matrix, which is consistent with the expected increase in computational complexity due to the quadratic rise in the number of operations with matrix size. Across all tested matrix sizes, the BLAS algorithm consistently has lower run times than the column-oriented algorithm, indicating the effectiveness of BLAS optimizations for these operations. The linear trend lines suggest that the BLAS algorithm is more scalable, showing a capability to handle larger matrices with a relatively smaller increase in run time. The results underscore the importance of using specialized mathematical libraries such as BLAS, particularly for large-scale computations, to significantly improve computational efficiency.

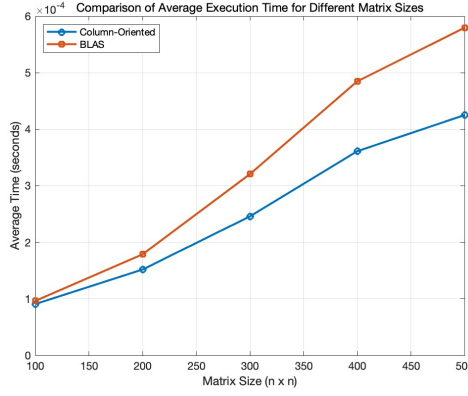


Figure 2: Comparison of Column-Oriented Sum with BLAS-Based Sum

3.2 Matrices multiplication

MATLAB's built-in matrix multiplication outperforms the manual approach, showing significant speed advantages, especially as matrix size increases. The scalability of the built-in function is evident as it maintains a low and stable average time for increasing matrix sizes, while manual multiplication time grows likely with $O(n^3)$ time complexity. The efficiency gains from MATLAB's optimized algorithms are clear, suggesting the use of built-in functions for computationally intensive tasks. For large matrices, the built-in functions are preferable due to their computational efficiency.

Algorithm 3 Matrix Multiplication

```

1: function NAIVEMATRIXMULTIPLICATION( $n$ )
2:   Generate two random  $n \times n$  matrices  $A$  and  $B$ 
3:   Initialize matrix  $C$  with zeros of size  $n \times n$ 
4:   for  $i$  from 1 to  $n$  do
5:     for  $j$  from 1 to  $n$  do
6:       for  $k$  from 1 to  $n$  do
7:          $C[i, j] := C[i, j] + A[i, k] \cdot B[k, j]$ 
8:       end for
9:     end for
10:  end for
11: end function

```

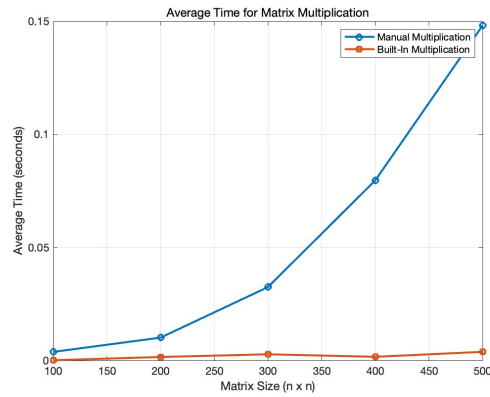


Figure 3: Comparison of two matrices multiplication

4 Exercise 4

4.1 LUP decomposition with partial pivoting strategy

Listing 3: LUP decomposition function

```
function [L, U, P] = lup_decomposition(A)
    % Input: A - Square matrix
    % Output: L - Lower triangular matrix
    %          U - Upper triangular matrix
    %          P - Permutation matrix

    [n, ~] = size(A);
    L = eye(n); % Initialize L as the identity matrix
    U = A;      % Initialize U as A
    P = eye(n); % Initialize P as the identity matrix

    for k = 1:n-1
        [~, pivot_row] = max(abs(U(k:n, k))); % Partial
            pivoting
        pivot_row = pivot_row + k - 1; % Adjust
            pivot_row index

        % Swap rows in U and L using pivot_row
        U([k, pivot_row], k:n) = U([pivot_row, k], k:n);
        L([k, pivot_row], 1:k-1) = L([pivot_row, k], 1:k
            -1);

        % Update P matrix
        P([k, pivot_row], :) = P([pivot_row, k], :);

        % Perform elimination to update U and L
```

```

        for i = k+1:n
            L(i, k) = U(i, k) / U(k, k);
            U(i, k:n) = U(i, k:n) - L(i, k) * U(k, k:n);
        end
    end
end

```

4.2 Test of the implementation

Matrix A: This is the original square matrix that was used for testing. It has dimensions 3x3 and is shown below:

$$A = \begin{bmatrix} 4 & 3 & 6 \\ 3 & 6 & 9 \\ 1 & -2 & 3 \end{bmatrix}$$

Matrix L: This is the lower triangular matrix obtained from the LUP decomposition. It is an $n \times n$ matrix where n is the size of the square matrix. It is shown below:

$$L = \begin{bmatrix} 1.0000 & 0 & 0 \\ 0.7500 & 1.0000 & 0 \\ 0.2500 & -0.7333 & 1.0000 \end{bmatrix}$$

Matrix U: This is the upper triangular matrix obtained from the LUP decomposition. It is also an $n \times n$ matrix and is shown below:

$$U = \begin{bmatrix} 4.0000 & 3.0000 & 6.0000 \\ 0 & 3.7500 & 4.5000 \\ 0 & 0 & 4.8000 \end{bmatrix}$$

Matrix P: This is the permutation matrix that represents the row swaps performed during the decomposition. It is an $n \times n$ identity matrix with rows permuted as needed. In this case, it remained the same as the identity matrix:

$$P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Verification: The correctness of the LUP decomposition was verified by multiplying the matrices P and A , which should result in the same matrix as $L \cdot U$.

$$PA = LU = \begin{bmatrix} 4 & 3 & 6 \\ 3 & 6 & 9 \\ 1 & -2 & 3 \end{bmatrix}$$

As shown in the verification section, both $P \cdot A$ and $L \cdot U$ are equal to the original matrix A , confirming the correctness of the decomposition.

4.3 Comparison with command lu

This figure 3 presents a performance comparison between a self-implemented LUP decomposition function and MATLAB's built-in `lu` function. The horizontal axis represents the size of the matrices ($n \times n$), while the vertical axis shows the average execution time (in seconds) for each method to perform the decomposition. It is observed that as the size of the matrix increases, the average execution time for the self-implemented `lup_decomposition(A)` function grows linearly, indicating that its run time increases with the size of the matrix. In contrast, MATLAB's `lu` function exhibits almost constant and significantly lower execution times across different matrix sizes, demonstrating superior performance stability and optimization. These results highlight the efficiency advantage of MATLAB's function when executing large-scale matrix decompositions.

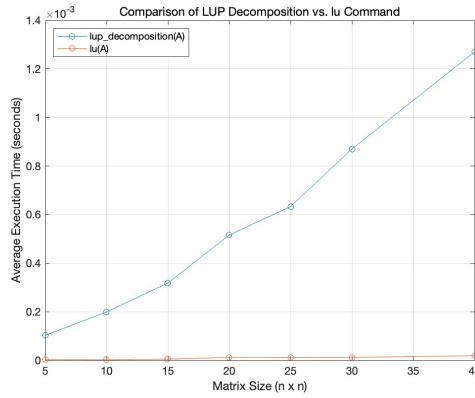


Figure 4: Comparison of LUP function with lu command